

Programación dinámica

Algoritmos y Estructura de Datos III

Michel J. Mizrahi

Facultad de Ciencias Exactas y Naturales,
Universidad de Buenos Aires

4 de Abril de 2012

Repaso

- ▶ Aplica a problemas de optimización donde obtenemos una solución utilizando una serie de decisiones.

Repaso

- ▶ Aplica a problemas de optimización donde obtenemos una solución utilizando una serie de decisiones.
- ▶ A medida que tomamos **decisiones**, subproblemas con la misma **forma** van surgiendo

Repaso

- ▶ Aplica a problemas de optimización donde obtenemos una solución utilizando una serie de decisiones.
- ▶ A medida que tomamos **decisiones**, subproblemas con la misma **forma** van surgiendo
- ▶ La **clave** es guardar el resultado de cada subproblema, en caso de que reaparezca

Repaso

- ▶ Aplica a problemas de optimización donde obtenemos una solución utilizando una serie de decisiones.
- ▶ A medida que tomamos **decisiones**, subproblemas con la misma **forma** van surgiendo
- ▶ La **clave** es guardar el resultado de cada subproblema, en caso de que reaparezca
- ▶ Es una combinación entre algoritmos **golosos** y algoritmos de **fuerza-bruta**

Repaso

- ▶ Al igual que Dividir y Conquistar, resuelve un problema combinando la solución óptima de varios subproblemas

Repaso

- ▶ Al igual que Dividir y Conquistar, resuelve un problema combinando la solución óptima de varios subproblemas
- ▶ Los subproblemas tienen un **solapamiento**.

Repaso

- ▶ Al igual que Dividir y Conquistar, resuelve un problema combinando la solución óptima de varios subproblemas
- ▶ Los subproblemas tienen un **solapamiento**.
- ▶ Cada subproblema es independiente. Solo interesan los resultados.

Repaso

- ▶ Al igual que Dividir y Conquistar, resuelve un problema combinando la solución óptima de varios subproblemas
- ▶ Los subproblemas tienen un **solapamiento**.
- ▶ Cada subproblema es independiente. Solo interesan los resultados.
- ▶ Optimalidad: Una solución óptima esta conformada por subsoluciones óptimas.

Cambio mínimo

Problema

Dados los valores de las monedas $V = \langle v_1, v_2, \dots, v_k \rangle$ y el monto a devolver, N . Quiero saber la mínima cantidad de monedas necesarias para devolver el cambio de manera exacta.

Cambio mínimo

Problema

Dados los valores de las monedas $V = \langle v_1, v_2, \dots, v_k \rangle$ y el monto a devolver, N . Quiero saber la mínima cantidad de monedas necesarias para devolver el cambio de manera exacta.

Ejemplo

$$V = \langle 3, 4, 5, 7 \rangle$$

$$N = 8$$

$$S = \langle 5, 3 \rangle$$

$$S = \langle 4, 4 \rangle$$

Caracterizar una solución

- ▶ Probar cualquier combinación de monedas.

Caracterizar una solución

- ▶ Probar cualquier combinación de monedas.
- ▶ Hasta cuantas veces uso cada moneda? Cuál sería la complejidad final?

Caracterizar una solución

- ▶ Probar cualquier combinación de monedas.
- ▶ Hasta cuantas veces uso cada moneda? Cuál sería la complejidad final?
- ▶ $O(2^{\dots})$. Feo de implementar, y costoso.

Pensar recursivamente

- Definamos la función *CambioMinimo*(N, V)

Pensar recursivamente

- ▶ Definamos la función *CambioMinimo*(N, V)
- ▶ El V es estático, nunca cambia, lo podemos olvidar por un rato...

Pensar recursivamente

- ▶ Definamos la función *CambioMinimo*(N , V)
- ▶ El V es estático, nunca cambia, lo podemos olvidar por un rato...
- ▶ Ahora pensemos si se puede resolver *CambioMinimo*(N) usando soluciones a instancias más chicas.

Pensar recursivamente

- ▶ Definamos la función *CambioMinimo*(N , V)
- ▶ El V es estático, nunca cambia, lo podemos olvidar por un rato...
- ▶ Ahora pensemos si se puede resolver *CambioMinimo*(N) usando soluciones a instancias más chicas.
- ▶ Puedo resolver *Cambio*(N) sabiendo *Cambio*(i), $i < N$?

Pensar recursivamente

Teorema

Sea $S = \langle s_1, \dots, s_m \rangle$ el cambio mínimo para N . Luego, $S_i = \langle s_1, \dots, s_i \rangle$ es óptimo.

Pensar recursivamente

Teorema

Sea $S = \langle s_1, \dots, s_m \rangle$ el cambio mínimo para N . Luego, $S_i = \langle s_1, \dots, s_i \rangle$ es óptimo.

Prueba

Supongamos que hay otra solución S^* mejor que S_i tal que $\sum S^* = \sum S_i$. Luego $|S^*| < |S_i|$ y $\sum S_i + \sum \langle s_{i+1}, \dots, s_m \rangle = \sum S^* + \sum \langle s_{i+1}, \dots, s_m \rangle = N$. Pero entonces $S' = S^* + \sum \langle s_{i+1}, \dots, s_m \rangle$ tiene longitud menor que S y $\sum S' = N$. Absurdo, pues S es mínimo por hipótesis.

Algoritmo-CambioMinimo

Algorithm 1: CambioMinimo

Data: $V = \langle v_1, \dots, v_k \rangle, N$

Result: $|S|$

if $res[N] > -1$ **then**

return $res[N]$

$res[N] = \infty$

for $i \leftarrow 1$ **to** k **do**

if $N \geq v_i$ **then**

$res[N] = \text{MIN}(res[N], 1 + \text{CambioMinimo}(N - v_i))$;

return $res[N]$

Algoritmo-CambioMinimo

Algorithm 2: CambioMinimo

Data: $V = \langle v_1, \dots, v_k \rangle, N$

Result: $|S|$

$c[0] = 0;$

for $i \leftarrow 1$ **to** N **do**

for $j \leftarrow 1$ **to** k **do**

$c[i] = \text{MIN}(c[i], c[i - v_j] + 1)$

return $c[N]$

Longest Common Subsequence (a.k.a. LCS)

Definición

Una subsecuencia de $S = \langle s_1, s_2, \dots, s_n \rangle$ es S con $k \geq 0$ elementos borrados.

Longest Common Subsequence (a.k.a. LCS)

Definición

Una subsecuencia de $S = \langle s_1, s_2, \dots, s_n \rangle$ es S con $k \geq 0$ elementos borrados.

Problema

Dados dos strings $X = \langle x_1, \dots, x_m \rangle$, $Y = \langle y_1, \dots, y_n \rangle$, encontrar $Z = \langle z_1, \dots, z_k \rangle$ tal que sea una subsecuencia de X e Y y tenga la mayor longitud posible.

Longest Common Subsequence (a.k.a. LCS)

Definición

Una subsecuencia de $S = \langle s_1, s_2, \dots, s_n \rangle$ es S con $k \geq 0$ elementos borrados.

Problema

Dados dos strings $X = \langle x_1, \dots, x_m \rangle$, $Y = \langle y_1, \dots, y_n \rangle$, encontrar $Z = \langle z_1, \dots, z_k \rangle$ tal que sea una subsecuencia de X e Y y tenga la mayor longitud posible.

Ejemplo

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

$Z = \langle B, C, B, A \rangle$

$Z = \langle B, D, A, B \rangle$



Caracterizar una solución

- Enumerar todas las posibles subsecuencias de X , ver si es subsecuencia de Y , y guardar la mejor.

Caracterizar una solución

- ▶ Enumerar todas las posibles subsecuencias de X , ver si es subsecuencia de Y , y guardar la mejor.
- ▶ Existen 2^m posibles subsecuencias de X

Caracterizar una solución

- ▶ Enumerar todas las posibles subsecuencias de X , ver si es subsecuencia de Y , y guardar la mejor.
- ▶ Existen 2^m posibles subsecuencias de X
- ▶ Funciona pero es muy costoso

Subestructuras óptimas

- Dada una secuencia S , llamamos $S_i = \langle s_1, \dots, s_i \rangle$

Teorema

Sea $Z = \langle z_1, \dots, z_k \rangle$ una LCS de $X = \langle x_1, \dots, x_m \rangle$ e $Y = \langle y_1, \dots, y_n \rangle$.

1. Si $x_m = y_n$ luego $z_k = x_m = y_n$ y $Z_{k-1} = LCS(X_{m-1}, Y_{n-1})$

Subestructuras óptimas

- Dada una secuencia S , llamamos $S_i = \langle s_1, \dots, s_i \rangle$

Teorema

Sea $Z = \langle z_1, \dots, z_k \rangle$ una LCS de $X = \langle x_1, \dots, x_m \rangle$ e $Y = \langle y_1, \dots, y_n \rangle$.

1. Si $x_m = y_n$ luego $z_k = x_m = y_n$ y $Z_{k-1} = LCS(X_{m-1}, Y_{n-1})$
2. Si $x_m \neq y_n$ luego $z_k \neq x_m$ implica Z es $LCS(X_{m-1}, Y)$

Subestructuras óptimas

- ▶ Dada una secuencia S , llamamos $S_i = \langle s_1, \dots, s_i \rangle$

Teorema

Sea $Z = \langle z_1, \dots, z_k \rangle$ una LCS de $X = \langle x_1, \dots, x_m \rangle$ e $Y = \langle y_1, \dots, y_n \rangle$.

1. Si $x_m = y_n$ luego $z_k = x_m = y_n$ y $Z_{k-1} = \text{LCS}(X_{m-1}, Y_{n-1})$
2. Si $x_m \neq y_n$ luego $z_k \neq x_m$ implica Z es $\text{LCS}(X_{m-1}, Y)$
3. Si $x_m \neq y_n$ luego $z_k \neq y_n$ implica Z es $\text{LCS}(X, Y_{n-1})$

Teorema

Sea $Z = \langle z_1, \dots, z_k \rangle$ una LCS de $X = \langle x_1, \dots, x_m \rangle$ e
 $Y = \langle y_1, \dots, y_n \rangle$

- ▶ Si $x_m = y_n$ luego $z_k = x_m = y_n$ y $Z_{k-1} = LCS(X_{m-1}, Y_{n-1})$

Teorema

Sea $Z = \langle z_1, \dots, z_k \rangle$ una LCS de $X = \langle x_1, \dots, x_m \rangle$ e $Y = \langle y_1, \dots, y_n \rangle$

- ▶ Si $x_m = y_n$ luego $z_k = x_m = y_n$ y $Z_{k-1} = \text{LCS}(X_{m-1}, Y_{n-1})$

Prueba

Si $z_k \neq x_m$, podemos extender cualquier Z con $x_m = y_n$ y obtener una subsecuencia común a ambas de longitud $k + 1$. Luego Z no es LCS. Por lo tanto $z_k = x_m = y_n$.

Z_{k-1} es una subsecuencia común de X_{m-1} e Y_{n-1} . Veamos que es una LCS. Supongamos que existe otra subsecuencia W tal que $|W| > k - 1$. Luego concatenando $x_m = y_n$ a W obtenemos W subsecuencia común de longitud mayor a k . Absurdo.

Teorema

Sea $Z = \langle z_1, \dots, z_k \rangle$ una LCS de $X = \langle x_1, \dots, x_m \rangle$ e $Y = \langle y_1, \dots, y_n \rangle$

- ▶ Si $x_m \neq y_n$ luego $z_k \neq x_m$ implica Z es LCS(X_{m-1} , Y)

Teorema

Sea $Z = \langle z_1, \dots, z_k \rangle$ una LCS de $X = \langle x_1, \dots, x_m \rangle$ e $Y = \langle y_1, \dots, y_n \rangle$

- Si $x_m \neq y_n$ luego $z_k \neq x_m$ implica Z es LCS(X_{m-1} , Y)

Prueba

Si $z_k \neq x_m$ luego Z es una LCS de (X_{m-1}, Y) . Si existiera $W = \text{LCS}(X_{m-1}, Y)$ con $|W| > k$, luego W seria LCS de X e Y con longitud mayor a k . Absurdo, pues entonces Z no es LCS.

Solución recursiva

- ▶ Por el teorema mencionado, en cada paso, debemos resolver uno o dos subproblemas
- ▶ Si $x_m = y_n$ buscamos $Z = LCS(X_{m-1}, Y_{n-1})$ y le concatenamos x_m
- ▶ Si $x_m \neq y_n$ buscamos $Z_1 = LCS(X_{m-1}, Y)$, $Z_2 = LCS(X, Y_{n-1})$ y devolvemos el mejor.
- ▶ Notar que $LCS(X_{m-1}, Y)$ y $LCS(X, Y_{n-1})$ comparten el subproblema $LCS(X_{m-1}, Y_{n-1})$

Solución recursiva

- Definamos $c(i, j)$ la longitud de $LCS(X_i, Y_j)$:

$$c(i, j) = \begin{cases} 0, & \text{if } i=0 \text{ or } j=0; \\ c(i-1, j-1) + 1, & \text{if } i, j > 0 \text{ and } x_i = y_j; \\ \max(c(i, j-1), c(i-1, j)), & \text{if } i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

- ▶ Escribimos la solución óptima en función de subsoluciones óptimas
- ▶ Demostramos la correctitud de la suboptimalidad
- ▶ Dimos una función recursiva que calcula el óptimo en función de los subóptimos
- ▶ Escribamos el algoritmo...

Algoritmo-LCS

Algorithm 3: Calcular-LCS

Data: $X = \langle x_1, \dots, x_m \rangle, Y = \langle y_1, \dots, y_n \rangle$

Result: Longitud de LCS

$c[0][1..n] = c[1..m][0] = 0;$

for $i \leftarrow 1$ **to** m **do**

for $j \leftarrow 1$ **to** n **do**

if $x_i = y_j$ **then**

$c[i][j] = c[i-1][j-1] + 1;$

else if $c[i-1][j] > c[i][j-1]$ **then**

$c[i][j] = c[i-1][j];$

else

$c[i][j] = c[i][j-1]$

return $c[m][n]$

Conclusiones

- ▶ Primero entender el problema, y pensar en la solución (aunque sea con algoritmos de fuerza-bruta)

Conclusiones

- ▶ Primero entender el problema, y pensar en la solución (aunque sea con algoritmos de fuerza-bruta)
- ▶ Pensar en funciones recursivas, y que parametros necesitarian.

Conclusiones

- ▶ Primero entender el problema, y pensar en la solución (aunque sea con algoritmos de fuerza-bruta)
- ▶ Pensar en funciones recursivas, y que parametros necesitarian.
- ▶ Ver que vale el principio de optimalidad.

Conclusiones

- ▶ Primero entender el problema, y pensar en la solución (aunque sea con algoritmos de fuerza-bruta)
- ▶ Pensar en funciones recursivas, y que parametros necesitarian.
- ▶ Ver que vale el principio de optimalidad.
- ▶ Demostrar correctitud del algoritmo.

Conclusiones

- ▶ Primero entender el problema, y pensar en la solución (aunque sea con algoritmos de fuerza-bruta)
- ▶ Pensar en funciones recursivas, y que parametros necesitarian.
- ▶ Ver que vale el principio de optimalidad.
- ▶ Demostrar correctitud del algoritmo.
- ▶ Hacer muchos ejercicios.